

■ Online Book Store

SQL Project – Database Design & Queries

■ Name: Neha Mudgal

■ Objective:

Design and implement a relational database system for an online book store and solve real-world business problems using SQL queries.

■ Key Features of the Project:

- Created Tables: • Books (book details) • Customers (customer details) • Orders (order tracking) -
Wrote 20+ SQL Queries to answer business questions - Practiced advanced SQL concepts: •
JOINS • Aggregations • Subqueries • Data Analysis

■ Looking forward to applying these skills in a Data Analyst role.

-- Create Tables

DROP TABLE IF EXISTS BOOKS;

```
CREATE TABLE BOOKS (  
    BOOK_ID SERIAL PRIMARY KEY,  
    TITLE VARCHAR(100),  
    AUTHOR VARCHAR(100),  
    GENRE VARCHAR(50),  
    PUBLISHED_YEAR INT,  
    PRICE NUMERIC(10, 2),  
    STOCK INT  
);
```

DROP TABLE IF EXISTS CUSTOMERS;

```
CREATE TABLE CUSTOMERS (  
    CUSTOMER_ID SERIAL PRIMARY KEY,  
    NAME VARCHAR(100),  
    EMAIL VARCHAR(200),  
    PHONE INT,  
    CITY VARCHAR(100),  
    COUNTRY VARCHAR(200)  
);
```

DROP TABLE IF EXISTS ORDERS;

```
CREATE TABLE ORDERS (  
    ORDER_ID SERIAL PRIMARY KEY,  
    CUSTOMER_ID INT REFERENCES CUSTOMERS (CUSTOMER_ID),  
    BOOK_ID INT REFERENCES BOOKS (BOOK_ID),  
    ORDER_DATE DATE,  
    QUANTITY INT,  
    TOTAL_AMOUNT NUMERIC(10, 2)  
);
```

--DATA HAVE BEEN IMPORTED IN ALL TABLES USING THE IMPORT/EXPORT DATA OPTION GIVEN
IN THE TABLE

```
SELECT * FROM BOOKS;
```

```
SELECT * FROM CUSTOMERS;
```

```
SELECT * FROM ORDERS;
```

--PROJECT ONLINE BOOK_STORE ASSIGNMENT ANSWERS

--1. Retrieve all the books in the 'Fiction' genre.

```
SELECT * FROM BOOKS  
WHERE GENRE = 'Fiction';
```

--2. Find the books published after the year 1950.

```
SELECT * FROM BOOKS  
WHERE PUBLISHED_YEAR > 1950  
ORDER BY PUBLISHED_YEAR;
```

--3. List all the customers from the Canada

```
SELECT * FROM CUSTOMERS  
WHERE COUNTRY = 'Canada';
```

--4. Show orders placed in november 2023.

```
SELECT * FROM ORDERS  
WHERE ORDER_DATE BETWEEN '01-11-2023' AND '30-11-2023'  
ORDER BY ORDER_DATE;
```

--5. Retrieve the total stocks of book available.

```
SELECT SUM(STOCK) AS TOTAL_STOCK  
FROM BOOKS;
```

--6. Find the details of the most expensive book.

```
SELECT *  
FROM BOOKS  
ORDER BY PRICE DESC  
LIMIT 1;
```

--7.Show all the customers name who ordered more than 1 quantity of a book.

```
SELECT  
O.ORDER_ID,
```

```
O.QUANTITY,  
C.NAME,  
C.CUSTOMER_ID  
FROM ORDERS O  
JOIN CUSTOMERS C ON O.CUSTOMER_ID = C.CUSTOMER_ID  
WHERE QUANTITY > 1  
ORDER BY QUANTITY;
```

--8.Retrieve all orders where the total amount exceeds \$20.

```
SELECT *FROM ORDERS  
WHERE TOTAL_AMOUNT > 20  
ORDER BY TOTAL_AMOUNT;
```

--9.List all the genre available in the books table.

```
SELECT DISTINCT(GENRE)  
FROM BOOKS;
```

--10.Find the book with the lowest stock.

```
SELECT  
    BOOK_ID,  
    TITLE,  
    STOCK  
FROM BOOKS  
ORDER BY STOCK  
LIMIT 1;
```

--11. Calculate the total revenue generated from all orders.

```
SELECT SUM(TOTAL_AMOUNT)
FROM ORDERS AS TOTAL_REVENUE;
```

--12. Retrieve the total number of books sold for each genre.

```
SELECT B.GENRE, SUM(O.QUANTITY) AS TOTTAL_BOOK_SOLD
FROM BOOKS B
JOIN ORDERS O
ON B.BOOK_ID = O.BOOK_ID
GROUP BY B.GENRE;
```

--13. Find the average price of books in the fantasy genre.

```
SELECT AVG(PRICE) AS AVG_PRICE
FROM BOOKS
WHERE GENRE = 'Fantasy';
```

--14. List customers who have placed at least 2 orders.

```
SELECT C.CUSTOMER_ID, C.NAME, COUNT(O.ORDER_ID) AS ORDER_COUNT
FROM CUSTOMERS C
JOIN ORDERS O
ON C.CUSTOMER_ID = O.CUSTOMER_ID
GROUP BY C.CUSTOMER_ID
HAVING COUNT(O.ORDER_ID) >= 2;
```

--15. Find the most frequently ordered book.

```
SELECT COUNT(O.ORDER_ID) AS ORDER_COUNT,  
       B.TITLE,  
       B.BOOK_ID  
FROM BOOKS B  
JOIN ORDERS O  
ON B.BOOK_ID = O.BOOK_ID  
GROUP BY B.BOOK_ID  
ORDER BY ORDER_COUNT DESC  
LIMIT 1;
```

--16. Show the top 3 most expensive books of 'fantasy ' genre.

```
SELECT *  
FROM BOOKS  
WHERE GENRE = 'Fantasy'  
ORDER BY PRICE DESC  
LIMIT 3;
```

--17. Retrieve the total quantity of books sold by each author.

```
SELECT B.AUTHOR, SUM(O.QUANTITY) AS TOTAL_BOOK_SOLD  
FROM BOOKS B  
JOIN ORDERS O  
ON B.BOOK_ID = O.BOOK_ID  
GROUP BY (B.AUTHOR);
```

--18. List the cities where customers who spent over \$30 are located.

```
SELECT DISTINCT C.CITY, TOTAL_AMOUNT
FROM CUSTOMERS C
JOIN ORDERS O
ON C.CUSTOMER_ID = O.CUSTOMER_ID
WHERE TOTAL_AMOUNT > 30;
```

--19. Find the customer who spent the most on orders.

```
SELECT
    O.CUSTOMER_ID,
    C.NAME,
    SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENT
FROM ORDERS O
JOIN CUSTOMERS C ON O.CUSTOMER_ID = C.CUSTOMER_ID
GROUP BY O.CUSTOMER_ID, C.NAME
ORDER BY TOTAL_SPENT DESC
LIMIT 1;
```

--20. Calculate the stock remaining after fulfilling all orders.

```
SELECT
    B.BOOK_ID,
    B.TITLE,
    B.STOCK,
    COALESCE(SUM(O.QUANTITY), 0) AS ORDER_QUANTITY,
```



```
B.STOCK - COALESCE(SUM(O.QUANTITY), 0) AS REMAINING_STOCK  
FROM BOOKS B  
LEFT JOIN ORDERS O  
ON B.BOOK_ID = O.BOOK_ID  
GROUP BY B.BOOK_ID;
```